

MLflow tracking

The MLflow tracking component serves the purpose of providing an API and UI. It is used for tracking the following items:

- Parameters
- Code versions
- Metrics
- Output files

These parameters are tracked during the execution of machine learning code and later used for visualisation purposes. MLflow tracking gives you the ability to track experiments using Python, Java, R and REST API.

MLflow projects

MLflow projects provide a format for packaging the machine learning projects that can be reused and reproduced later. This component includes an API and command-line tools that can be used to execute projects. This helps to chain together projects into workflows.

MLflow models

MLflow models provide a standard format for machine learning model packaging. The models can be packaged in a manner that they can later be used with various downstream tools. This enables real-time serving via REST API or batch inference using Apache Spark.

MLflow model registry

The MLflow model registry serves the purpose of providing a centralised store for models, a set of APIs and UI. These are used to collaboratively manage the life cycle of an MLflow model. It enables the following:

- Model lineage
- Model versioning
- Transitions from staging to production
- Annotations

MLflow installation

MLflow can be installed easily. For example, in Python this can be done using the PIP command:

```
pip install mlflow
```

With R, it can be installed as follows:

```
install.packages("mlflow")
mlflow::install_mlflow()
```

To go ahead with quickstart, the code at <https://github.com/mlflow/mlflow> can be cloned.

Tracking an API

With MLflow tracking API, logging of parameters, metrics and artifacts can be carried out easily. A simple example is given below:

```
import os
from random import random, randint
from mlflow import log_metric, log_param, log_artifacts

if __name__ == "__main__":
    # Log a parameter (key-value pair)
    log_param("sample1", randint(0, 100))

    # Log a metric; metrics can be updated throughout the run
    log_metric("Test", random())
    log_metric("Test", random() + 1)
    log_metric("Test", random() + 2)

    # Log an artifact (output file)
    if not os.path.exists("outputs"):
        os.makedirs("outputs")
    with open("outputs/osfy.txt", "w") as f:

        f.write("Welcome to MLflow")
    log_artifacts("outputs")
```

The tracking UI

The MLflow tracking UI can be invoked with the following command:

```
mlflow ui
```

With the successful execution of the above command, the tracking UI can be launched in the browser with the following URL: <http://localhost:5000>.

Executing MLflow projects

As stated earlier, MLflow can be used to package code and the associated dependencies into a project. The saved projects can be executed with the MLflow *run* command. The projects can be located from the local folder or from GitHub. A sample is shown below:

```
mlflow run sklearn_elasticnet_wine -P alpha=0.5
```

```
mlflow run https://github.com/mlflow/mlflow-example.git -P alpha=5.0
```

An example using a linear regression model

Let us explore a complete example with a simple linear regression model. Carry out the following steps to get the environment ready.

- Install MLflow and Scikit-learn (if it is not already installed in your system):

```
pip install mlflow[extras]
```

- Install Conda.
- From the MLflow repository, clone the code:

```
git clone https://github.com/mlflow/mlflow
```

- Navigate to the *examples* folder. The training code is located at *examples/sklearn_elasticnet_wine/train.py*.

A portion of the code is shown below. (The complete code is available at <https://mlflow.org/docs/latest/tutorials-and-examples/tutorial.html>.)

```
with mlflow.start_run():

    lr = ElasticNet(alpha=alpha, l1_ratio=l1_ratio, random_state=42)
    lr.fit(train_x, train_y)

    predicted_qualities = lr.predict(test_x)
```